

TAGS — the mechanic vocabulary, and the rule for which word to use

Opened 2026-08-04. Will: "Idk the diff between flags and tags formally u design it like an industry pro would." They had been used interchangeably in conversation while meaning two different things in the code. This file names them.

The pattern is a **capability / handler registry**: a move declares *what kind of thing it is*, an ability or type or item declares *what it does to that kind of thing*, and a resolver joins the two. Nothing special-cases Fake Out.

The three words

	FLAG	TAG	PARAM
Who owns it	Pokémon Showdown	ABRA	ABRA
Where it lives	<code>move.flags.*</code> in the dex	<code>data/tags.json</code> → <code>data/abra-tags.js</code>	inside a tag
Who writes it	Game Freak, via Showdown	<code>engine/tag_dex.js</code>	<code>engine/tag_dex.js</code>
Shape	boolean	a name on a list	an object
Examples	<code>contact</code> , <code>powder</code> , <code>sound</code> , <code>bullet</code> , <code>charge</code> , <code>reflectable</code>	<code>redirects</code> , <code>drain</code> , <code>multiHit</code> , <code>blocksMove</code> , <code>swapsStat</code> , <code>immuneToMoveClass</code>	<code>{what:'priority', priorityAbove:0}</code>
May we invent one	No.	Yes — that is the point	Yes

A FLAG is a fact about the game. It is upstream, canonical, and boolean. `contact` is true of Fake Out because Game Freak said so. We do not get a vote, and we never author one.

A TAG is our classification of what something DOES. It is derived, it carries parameters, and it is the vocabulary the engine actually speaks. `redirects` is not a Showdown concept; it is ABRA saying *this move retargets an attack*, with a param describing how.

A PARAM is the tag's payload — the numbers and conditions a consumer needs. `blocksMove` alone says "this ability refuses something"; `{what:'priority', priorityAbove:0}` says which.

The rule for choosing

*Is this something the game declares, or something we concluded? Declared → it is a flag, and you read it through a tag. Concluded → it is a tag, and it must be **derived** in `tag_dex.js`, never typed.*

Direction of dependency — one way, and it matters



`engine/tag_dex.js:768` is the join, and it is worth reading because it is the whole design in one line:

```
of: m => (m.flags && m.flags.contact) ? { contact: true } : null
```

A tag named `contact` **derived from** a flag named `contact` . The names match; the layers do not.

Consumers read TAGS, not flags. A consumer that reaches past the tag layer into `m.flags` directly has forked the vocabulary, and the two copies will disagree eventually — silently, because both keep working. The one declared exception is `engine/board.js:1924` , which owns the powder question for its own feature path and says so in a comment; that is a stated exception, not a precedent.

The interaction model

Three participants, joined at run time rather than enumerated at write time:

1. **The move declares a capability** — its tags. Fake Out is `[priority, contact, statusInflict, flinches]` .
2. **Abilities, types, items and field states declare reactions**, keyed by capability shape. Armor Tail is `blocksMove {what:'priority'}` . Rough Skin is `punishesAttacker + contactPunish` . Bulletproof is `immuneToMoveClass {blocksFlag:'bullet'}` .
3. **A resolver joins them by SHAPE, not by name.**

```
// engine/medicham2-browser.js:120 - the pattern to copy
for (const id of TAGS.withTag('ability', 'blocksMove')) {
  const p = TAGS.param('ability', id, 'blocksMove');
  if (p && p.what === 'priority') _prioBar.set(id, p.priorityAbove ?? 0);
}
```

Derived, not named. An ability added to the artifact next month with the same tag shape is picked up without editing the engine. An engine that says `if (ability === 'armortail')` is a list that someone has to remember to extend, and nobody will.

`TAGS.reactorsTo(capability)` is the reverse index — everything that reacts to a given capability.

Resolution order, and why the stage decides the failure mode

This is the general form of "*what happens when I click Fake Out*". **The stage a mechanic resolves at determines what failure looks like**, and collapsing the stages is how the Sucker Punch bug survived.

#	Stage	Question	Failure looks like
1	Legality	may this be chosen at all?	removed from the menu — Fake Out after turn 1, a Choice-locked move, a blocked priority move
2	Targeting	who does it actually hit?	retargeted — redirection; or not retargeted , when the redirector is a Grass type and the move is <code>powder</code>
3	Immunity	does it connect?	zero damage — type chart, <code>typeImmunity</code> , <code>immuneToMoveClass</code>
4	Damage	how much?	a different number — STAB, weather, <code>swapsStat</code> , <code>multiHit</code> , <code>drain</code>
5	Secondary	what else happens?	effect suppressed — a flinch on an immune target is not applied

Three consequences that are not obvious:

- **A blocked priority move FAILS, it does not lose the speed tie.** So it is dropped from the menu at stage 1, never reordered at stage 2. `priorityRefusedAbove()` returns the highest priority that still resolves; `Infinity` means nothing is refused.
- **An immune target takes nothing — not the damage, and not the secondary.** Stage 3 short-circuits stage 5 (commit `e8deb23`).

- **Ordering matters inside a stage.** A flinch only stops a target that has not already moved, so move order resolves before the flinch applies — otherwise a slow Rock Slide flinches someone who already attacked.

Worked example — why the PARAM is the fact and the NAME is only a label

Spicy Extract (Hydrapple's signature) gives the *target* **+2 Attack and -2 Defense**. It is one of the few moves that hands your opponent a buff on purpose.

Its tag list reads:

```
tags: ["neverMisses", "lowersTarget", "statusCategory", "statChange"]
params: lowersTarget { readFrom:"m.boosts", lowersSpeed:false, lowersAttack:false }
      statChange { target: [{ boosts: {atk: 2, def: -2}, chance: 100 }] }
```

`lowersTarget` **is a misleading name for this move.** A consumer keying off the tag *name* would read "this is a debuff" and miss that it grants +2 Attack — which is the whole reason anybody clicks it. The name is wrong and it does not matter, because the **param** carries `{atk: +2, def: -2}` and `engine/medicham2-browser.js:2168` reads the param:

```
const _sc = TAGS.param('move', id, 'statChange');
```

So the move is modelled correctly, including the half that helps the opponent. `:2134` already contrasts the two in a comment.

The rule: a tag NAME is a searchable label and may be imprecise. A tag PARAM is a fact and must not be. **Consumers key off params.** A consumer that branches on a tag name has turned a label into a fact, and the first move whose name does not describe it will be silently wrong.

Invariants, each with the bug it exists to prevent

1. Every tag has a consumer, or is declared unconsumed with a reason. A tag nothing reads is a silent no-op and the engine keeps reporting success. This project's signature bug, three times: `blocksMove` on Armor Tail read only by `clickFragility`'s bench check — Sucker Punch beat a Farigiraf in every rollout ever run; `swapsStat` on Foul Play where `grep` returned zero — 734 uses, 1.55× too high; `immuneToMoveClass` on Bulletproof, also zero. `tag_dex.js` already checks part of this ("*and check whether anything actually reads them*"). Finishing it is PRIORITIES #42.

2. A fact has exactly one implementation. Two files that both decide Choice Scarf multiplies Speed by 1.5 will disagree eventually, invisibly, because both keep working. `choiceLock` was tagged and `board.js` passed its own test while `medicham2` did not — two engines disagreeing about a fact.

3. Membership is derived, never typed. Match on tag shape. A hand-written list of ability names is a list someone must remember to extend.

4. Print what a derivation matched, before using it. Every derived set in this project over-matched on its first attempt — `refusesStatusMoves` caught Telepathy and Wonder Guard, `speedOnItemLoss` caught Sticky Hold. `test-engine-diff.js` prints its derived set every run for exactly this reason.

5. A probe is staged against a body that can actually show the effect. The redirection "bug" of 2026-08-04 was a probe firing **Dragon** Claw at a **Fairy** type: the redirect worked, moved the attack onto something immune, and both arms read 0. Nine probes have now been wrong before the engine was.

Adding a mechanic — the checklist

1. **Decide the layer.** Declared by the game → read the existing flag. Concluded by us → a new tag.
2. **Derive it in** `tag_dex.js` . Never hand-write an entry into `data/tags.json` .
3. **Print what it matched** and read the list. Assume it over-matched.
4. **Write the probe in** `tests/test-mechanics.js` **and WATCH IT FAIL.** A probe written after the fix tests the fix, not the mechanic. Stage it against a target that can show the effect.
5. **Write the consumer**, matching on shape, in the one place that owns that fact.
6. **Re-run the census** — it must not go down — **and the seeded differential** at the canonical seed, which must not get worse.
7. **If it makes a currently-green comparison red**, land the fix and the harness change in the **same pass**. A red test with no fix beside it is banned.